

Contents

guide_01_parameter_efficient_transfer_learning_for_nlp	1
0. 这篇论文一句话	2
1. 历史语境	2
2. 论文结构地图	3
3. 核心概念	4
3.1 Feature-based transfer	4
3.2 Full fine-tuning	4
3.3 Adapter tuning	4
3.4 Bottleneck adapter	5
3.5 Near-identity	5
3.6 Layer norm and task head	6
4. Figure 1: 参数效率权衡	6
5. Figure 2: Adapter 放在哪里	6
6. Adapter 数学	7
7. 参数量直觉	8
8. Near-identity 为什么重要	9
9. 训练过程	9
10. Table 1: GLUE 主结果	10
11. Table 2: 额外 17 个分类任务	10
12. Figure 3 and Figure 4: 参数-性能曲线	11
13. Figure 5: SQuAD	11
14. Figure 6: Ablation and initialization	12
15. 为什么不是只 fine-tune top layers	12
16. 为什么不是只训练 layer norm	12
17. 本地代码映射	13
18. 最小代码片段	14
19. 30 到 60 分钟本地实验	14
20. 用 AI agent 正确学习这篇	15
21. 与 Prefix-Tuning 和 LoRA 的关系	16
22. 常见误解	16
23. 局限性	17
24. 现代意义	17
25. 闭卷掌握检查	18
26. 最小复述模板	18

guide_01_parameter_efficient_transfer_learning_for_nlp

原论文: Parameter-Efficient Transfer Learning for NLP

本地原文 PDF:

learning/adapter-tuning-family/paper/01_parameter_efficient_transfer_learning_for_nlp.pdf

作者: Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, Sylvain Gelly

机构: Google Research, Jagiellonian University

arXiv: 1902.00751

会议: ICML 2019

类型: adapter tuning, parameter-efficient transfer learning, BERT adaptation

0. 这篇论文一句话

这篇论文提出把小型 adapter modules 插入 pretrained Transformer, 冻结原始 BERT 参数, 每个任务只训练少量 adapter、layer norm 和分类头参数。

它要解决的问题不是 “BERT 能不能微调”, 而是:

如果任务不断增加,
我们是否必须为每个任务保存一整份 BERT?

Houlsby adapters 的答案是:

```
one frozen shared backbone
+ small task-specific adapter modules
```

论文的核心证据是:

- GLUE 上 adapter 平均分 80.0, full fine-tuning 80.4。
- GLUE 多任务总存储从约 9 full BERT copies 降到约 1.3 copies。
- 额外 17 个分类任务上也接近 fine-tuning。
- SQuAD v1.1 上 adapter size 64 达到 90.4 F1, fine-tuning 约 90.7。

所以这篇论文是 PEFT 的重要起点之一: 它把 “冻结大模型, 只训练小模块” 做成了可扩展、可插拔的工程范式。

1. 历史语境

2018 到 2019 年, BERT 让 NLP 迁移学习进入一个新阶段。

常见流程变成:

```
pretrain BERT on large corpus
fine-tune all BERT weights on downstream task
deploy one fine-tuned model per task
```

这在单任务上非常有效。但论文关注的是 online / streaming task setting:

```
task_1 arrives
task_2 arrives
task_3 arrives
...
```

如果每个新任务都复制并微调整个 BERT, 系统会遇到几个问题:

- 存储随任务数量线性增长, 每个任务是一整份模型。
- 新任务增加时, 需要管理许多独立模型副本。
- 不能自然做到 compact model。
- 不能自然做到 extensible model。
- 多任务同时训练需要同时访问所有数据, 不适合任务按时间到达。

论文里的两个关键词很重要。

Compact:

解决很多任务时，每个任务只增加很少参数。

Extensible:

新任务可以后来加入，不需要重新访问或重训旧任务。

Adapter-tuning 的设计正是围绕这两个词。

2. 论文结构地图

读原文建议这样走：

Abstract:

- 说明 full fine-tuning 在多任务下参数效率低。
- 提出 adapter modules。
- 说明 GLUE 上只差 0.4%，每任务只训练 3.6% 参数。

Figure 1:

- 画出 accuracy delta 与 trainable parameters per task 的权衡。
- Adapter 曲线在更少参数下接近 full fine-tuning。

Section 1 Introduction:

- 引入 streaming downstream tasks。
- 对比 feature-based transfer、fine-tuning、adapter tuning。
- 强调 compact and extensible downstream models。

Section 2 Adapter tuning for NLP:

- 给出 adapter tuning 的抽象。
- 提出 bottleneck adapter。
- 说明 near-identity initialization。
- 说明为什么 frozen shared backbone 可以避免 forgetting。

Section 2.1 Instantiation for Transformer Networks:

- Figure 2 是核心图。
- 每个 Transformer layer 插两个 adapters。
- 一个在 attention sub-layer 之后。
- 一个在 feed-forward sub-layer 之后。
- 训练 adapter、layer norm、final classifier。

Section 3 Experiments:

- 3.2 GLUE。
- 3.3 额外 17 个 text classification tasks。
- 3.4 参数-性能 trade-off。
- 3.5 SQuAD。
- 3.6 Analysis and Discussion。

Figure 6:

- Adapter ablation。
- Initialization scale。
- 哪些层更重要。

Related Work:

- Pre-trained representations.
- Fine-tuning.
- Multi-task learning.
- Continual learning.
- Vision adapters.
- Concurrent BERT PALs.

3. 核心概念

3.1 Feature-based transfer

Feature-based transfer 是:

```
pretrained encoder extracts features
downstream model consumes features
```

优点:

- 原 pretrained network 不动。
- 多任务可以共享 encoder。

缺点:

- 下游模型只能读 features。
- 不能改变 pretrained network 内部的信息处理过程。

论文把它抽象成:

$$\text{new task model} = g_v(f_w(x))$$

其中 w 是 pretrained 参数, v 是下游模型参数。

3.2 Full fine-tuning

Full fine-tuning 是:

```
copy pretrained parameters w
train all w on downstream task
store one new copy per task
```

优点:

- 表达能力强。
- 当时是 BERT 下游任务的强基线。

缺点:

- 每任务训练 100% 参数。
- 每任务保存一整份模型。
- 任务多时存储成本很高。

3.3 Adapter tuning

Adapter tuning 是:

```
freeze w
insert small task-specific modules v
train v
```

抽象地说:

$\text{adapted model} = f_{\{w, v\}}(x)$

初始化时希望:

$f_{\{w, v_0\}}(x)$ is close to $f_w(x)$

也就是新插入的 adapter 一开始几乎不改变原模型。这就是 near-identity initialization。

3.4 Bottleneck adapter

论文的 adapter 是一个小 MLP:

input h : dimension d

down projection:
 $d \rightarrow m$

nonlinearity:
 $f(\dots)$

up projection:
 $m \rightarrow d$

residual:
 $\text{output} = h + \text{adapter_delta}$

其中 m 是 bottleneck dimension, 远小于 hidden dimension d 。

所以参数量大约是:

$d * m + m * d$

而不是一个完整 $d * d$ 矩阵。

3.5 Near-identity

Adapter 插进一个已经很强的 pretrained network。如果随机初始化太大, 一开始就会扰乱原模型, 训练可能不稳定。

因此论文要求 adapter 初始化接近 identity。

常见实现方式是:

```
up projection initialized near zero
adapter_delta near zero
output near input h
```

这样训练第一步时, 模型行为接近原始 BERT。

3.6 Layer norm and task head

论文不是只训练 adapter。

Figure 2 caption 说明，下游训练时的绿色部分包括：

- adapter modules.
- layer normalization parameters.
- final classification layer.

这点容易被忽略。如果你只说“只训练 adapter”，是不完整的。

4. Figure 1: 参数效率权衡

Figure 1 的横轴是每任务训练参数量，纵轴是相对 full fine-tuning 的 accuracy delta。

图想表达：

full fine-tuning:

train many parameters per task
performance strong

fine-tune top layers:

train fewer parameters
performance drops quickly on GLUE

adapters:

train far fewer parameters
stay close to full fine-tuning

这张图的重点不是某个单点分数，而是曲线：

accuracy vs trainable parameters per task

读这篇论文时，始终要盯住这个二维权衡。

5. Figure 2: Adapter 放在哪里

Transformer layer 里有两个主要 sub-layers:

multi-head attention
feed-forward network

Houlsby adapter 在每个 Transformer layer 放两个 adapter:

```
input hidden
|
v
multi-head attention sub-layer
|
v
adapter 1
|
v
layer norm / residual path
```

```

    |
    v
feed-forward sub-layer
    |
    v
adapter 2
    |
    v
layer norm / residual path

```

更抽象一点:

Transformer layer 1

attention branch:

```

h_attn = Attention(...)
h_attn = Adapter_attn(h_attn)

```

feed-forward branch:

```

h_ffn = FFN(...)
h_ffn = Adapter_ffn(h_ffn)

```

这就是为什么本仓库里 `houlsby_minimal.py` 的 trainable adapter 数量 大约是 Pfeiffer 单 adapter 变体的两倍。

6. Adapter 数学

给定 hidden state:

$h \in \mathbb{R}^d$

adapter 计算:

```

z = W_down h + b_down
z = activation(z)
delta = W_up z + b_up
out = h + delta

```

其中:

```

W_down: shape (m, d)
W_up:   shape (d, m)
m:      bottleneck dimension
d:      hidden dimension

```

单个 adapter 参数量:

```

down weights: d * m
down bias:    m
up weights:   m * d
up bias:      d

```

```

single adapter params = d*m + m + m*d + d
                      = 2*d*m + d + m

```

Houlsby 每层两个 adapter:

$\text{params per layer} = 2 * (2*d*m + d + m)$

全模型 task-specific adapter 参数:

$\text{total adapter params} = L * 2 * (2*d*m + d + m)$

其中 L 是 Transformer layer 数。

7. 参数量直觉

以 GPT-2 base 量级为例:

$L = 12$

$d = 768$

$m = 16$

adapters per layer = 2

单个 adapter:

$2*d*m + d + m$

$= 2*768*16 + 768 + 16$

$= 25360$

每层两个:

50720

12 层:

608640

这就是本仓库无依赖文件会打印的数字。

对 BERT large 量级, adapter 参数仍然远少于 full fine-tuning。

论文 Table 1 更关注多任务总存储:

GLUE tasks:

full fine-tuning total: about 9x BERT

adapters total: about 1.3x BERT

含义:

full fine-tuning:

task_1 full BERT

task_2 full BERT

...

adapter tuning:

one shared frozen BERT

task_1 adapters

task_2 adapters

...

8. Near-identity 为什么重要

如果 adapter 初始输出很大，它会在训练开始时破坏 BERT 的 hidden states。

理想情况是：

```
adapter_delta = 0
out = h
```

于是插入 adapter 后，模型初始行为和原模型几乎一致。

本仓库的玩具代码：

```
learning/adapter-tuning-family/src/adapter_original_minimal.py
```

用 zero up projection 演示这件事。

如果 $W_{up} = 0$ 且 $b_{up} = 0$ ，那么：

```
delta = 0
out = h
```

这对应论文 Section 2 对 near-identity initialization 的要求，也对应 Figure 6 右侧的初始化尺度实验：初始化太大时性能下降。

9. 训练过程

Adapter-tuning 的训练流程：

1. Load pretrained BERT.
2. Insert adapter modules into each Transformer layer.
3. Initialize adapters close to identity.
4. Freeze original BERT weights.
5. Add task-specific classification or QA head.
6. Train:
 - adapters
 - layer norm parameters
 - task head
7. Save:
 - shared BERT once
 - one adapter set per task

和 full fine-tuning 的关键差别：

```
full fine-tuning updates w
adapter-tuning keeps w fixed and updates v
```

和 multi-task learning 的关键差别：

```
multi-task learning needs tasks together
adapter-tuning can add tasks sequentially
```

和 continual learning 的关键差别：

```
continual learning modifies shared weights and may forget
adapter-tuning freezes shared weights and keeps old adapters
```

10. Table 1: GLUE 主结果

论文用 BERT large 在 GLUE 上评估。WNLI 按 BERT 论文惯例省略。

Table 1 的关键数字:

Full fine-tuning:

total params: 9.0x
trained params per task: 100%
mean GLUE score: 80.4

Adapters with task-specific sizes:

total params: 1.3x
trained params per task: 3.6%
mean GLUE score: 80.0

Adapters fixed size 64:

total params: 1.2x
trained params per task: 2.1%
mean GLUE score: 79.6

这张表证明:

Adapter tuning 在 GLUE 上几乎追平 full fine-tuning, 但多任务总参数从 9 倍降到约 1.3 倍。

注意不要误读:

这不是说 adapter 每个单任务都必然比 full fine-tuning 高。它证明的是总体权衡很强:

almost same average score
much lower task-specific parameter cost

11. Table 2: 额外 17 个分类任务

为了避免只在 GLUE 上成立, 论文又收集了 17 个公开分类任务。

这些任务差异很大:

- 训练样本从约 900 到 330k。
- 类别数从 2 到 157。
- 文本长度差异明显。

它们用 BERT base 评估。

Table 2 的关键信息:

No BERT AutoML baseline average: 72.7
BERTBASE fine-tune average: 73.7
Variable fine-tune average: 74.0
Adapters average: 73.3

Full fine-tuning total params: 17x
Variable fine-tuning total: 9.9x
Adapters total: 1.19x
Adapters trained per task: 1.14%

这说明 adapter 不只是 GLUE trick。在更杂的任务集合上，它仍然用极小的额外存储接近 fine-tuning。也要看到限制：

有些任务 adapter 明显不如其他方法，例如 SMS spam collection 上 adapter 波动较大。所以这不是“所有任务无损替代”。

12. Figure 3 and Figure 4: 参数-性能曲线

Figure 3 聚合 GLUE 和额外任务，比较：

- adapters of different sizes.
- fine-tuning top k layers.

结论：

adapters keep good performance with far fewer trainable parameters
top-layer fine-tuning often drops much faster

Figure 4 在 MNLI_m 和 CoLA 上放大细节，加入了只训练 layer norm 的 baseline。

关键观察：

- 只训练 layer norm 参数很少，但性能差。
- 只 fine-tune top layers 在相同参数预算下不如 adapters。
- Adapter size 64 在 MNLI_m 上约 2M trainable params，性能约 83.7。
- 只 fine-tune top layer 约 9M trainable params，性能约 77.8。

这解释了 adapter 的机制价值：

不是“可训练参数少”本身带来效果，而是“少量参数被放在每层内部正确位置”带来效果。

13. Figure 5: SQuAD

论文还在 SQuAD v1.1 extractive QA 上测试。

任务形式：

input:

```
question
paragraph
```

output:

```
answer span start/end
```

结果：

- Full fine-tuning F1 约 90.7。
- Adapter size 64 约 90.4 F1，训练约 2% 参数。
- Adapter size 2 也能达到约 89.9 F1，只训练约 0.1% 参数。

这说明 adapter 不只适合分类，也适合 span extraction。

但也要注意：

论文没有覆盖生成任务。这和 Prefix-Tuning 论文的任务重心不同。

14. Figure 6: Ablation and initialization

Figure 6 左侧和中间:

作者训练好 adapter 后, 不重新训练, 直接移除连续 layer spans 的 adapters, 看验证性能下降。

观察一:

```
remove one layer's adapters:  
    largest drop is around 2%
```

观察二:

```
remove all adapters:  
    MNLI drops to about 37%  
    CoLA drops to about 69%
```

这说明每个 adapter 的单独影响小, 但整体影响很大。

观察三:

低层 adapters 影响较小, 高层 adapters 更重要。

作者解释:

- 低层更像共享低级特征。
- 高层更任务相关。

Figure 6 右侧:

初始化尺度实验显示, 较小初始化比较稳, 初始化太大会明显伤害性能。

这支持 near-identity 设计。

15. 为什么不是只 fine-tune top layers

直觉上, 如果高层更任务相关, 那只微调 top layers 是否足够?

论文的 Figure 3/4 回答:

在相同参数量附近, adapter 往往比 top-layer fine-tuning 更好。

原因可以这样理解:

Top-layer fine-tuning:

```
only top layers can change  
lower/middle layers fixed
```

Adapters:

```
every layer has a small writable branch  
original path remains stable
```

也就是说 adapter 给每层都开了一个小的任务增量通道, 而不是只修改最后几层。

16. 为什么不是只训练 layer norm

Layer norm 参数非常少。训练它看起来很便宜。

但 Figure 4 显示:

只训练 layer norm 表现明显差。

这说明：

parameter-efficient does not mean parameter-minimal.

太少参数、放在不够表达的位置，无法有效适配任务。

Adapter 的优势是：

- 参数仍然少。
- 但每层有可学习的 nonlinear bottleneck branch。
- 可以改变 hidden processing。

17. 本地代码映射

本专题有很多后续 adapter 家族代码。这篇原论文最直接对应：

```
learning/adapter-tuning-family/src/houlsby_minimal.py
learning/adapter-tuning-family/src/pfeiffer_minimal.py
learning/adapter-tuning-family/src/tests/test_houlsby_pfeiffer.py
```

这几个文件会加载 GPT-2，演示：

- Houlsby 每个 block 两个 adapters。
- Pfeiffer 每个 block 一个 adapter。
- Houlsby trainable 参数是 Pfeiffer 的两倍。
- zero-up initialization 让初始 forward 接近 base。
- mini training 里 loss 能下降。

我为本篇 guide 新增了无依赖机制文件：

```
learning/adapter-tuning-family/src/adapter_original_minimal.py
learning/adapter-tuning-family/src/tests/test_adapter_original_minimal.py
```

它只复现原论文机制：

- single_adapter_parameters: 单个 bottleneck adapter 参数量。
- houlsby_adapter_parameters: Houlsby 两 adapter per layer 总参数。
- near_identity_demo: zero-up projection 下输出等于输入。
- adapter_total_params: 多任务 adapter 存储量。
- full_finetune_total_params: 多任务 full-copy 存储量。

建议先跑：

```
.\.venv\Scripts\python.exe `
learning\adapter-tuning-family\src\adapter_original_minimal.py
```

再跑：

```
.\.venv\Scripts\python.exe `
learning\adapter-tuning-family\src\tests\test_adapter_original_minimal.py
```

注意：

本机 .venv 中 adapter minimal 路径可用。adapters 第三方库路径是 optional，环境自检会跳过未安装的 adapters 库，避免为了可选库降级 transformers 而破坏其他专题。

18. 最小代码片段

下面是论文机制的极简版:

```
from adapter_original_minimal import AdapterConfig
from adapter_original_minimal import houlby_adapter_parameters
from adapter_original_minimal import single_adapter_parameters

print(single_adapter_parameters(hidden_dim=768, adapter_dim=16))

cfg = AdapterConfig(
    layers=12,
    hidden_dim=768,
    adapter_dim=16,
)

print(houlby_adapter_parameters(cfg))
```

应得到:

```
25360
608640
```

解释:

- 25360 是单个 down-up bottleneck adapter 参数量。
- 608640 是 12 层、每层两个 adapters 的总参数量。

near-identity:

```
from adapter_original_minimal import near_identity_demo

x, y = near_identity_demo()
print(x)
print(y)
```

应看到 $x == y$ 。

这对应论文的稳定训练直觉:

```
adapter starts as almost identity
then learns task-specific changes
```

19. 30 到 60 分钟本地实验

第一步:

```
.\.venv\Scripts\python.exe `
learning\adapter-tuning-family\src\adapter_original_minimal.py
```

你要解释:

- single_adapter_gpt2_r16 为什么是 25360。
- houlby_gpt2_r16 为什么是 608640。
- adapter_9_tasks 为什么远小于 full_finetune_9_tasks。
- near_identity_input 为什么等于 near_identity_output。

第二步:

```
.\.venv\Scripts\python.exe `
learning\adapter-tuning-family\src\tests\test_adapter_original_minimal.py
```

第三步, 如果想看真实 GPT-2 adapter:

```
.\.venv\Scripts\python.exe `
learning\adapter-tuning-family\src\houlsby_minimal.py
```

观察:

- trainable 参数量。
- 每个 adapter 的 down/up 参数分解。
- 初始 forward 与 base 的最大误差。

第四步, 做一个小改动:

把 adapter_original_minimal.py 里的:

```
adapter_dim = 16
```

改成:

```
adapter_dim = 32
```

重新计算参数量。

你应该观察到:

- single adapter 参数大约翻倍。
- Houlsby 总 adapter 参数也大约翻倍。

这对应论文 Figure 3/4:

adapter size 控制性能和参数量之间的 trade-off。

20. 用 AI agent 正确学习这篇

不要让 agent 直接总结。让 agent 逼你画机制图、算参数量、解释证据链。

可以这样提示:

我正在学习 Houlsby adapters 论文。

请一次只问一个问题, 按以下顺序考我:

1. full fine-tuning 在多任务场景下为什么参数效率低?
2. compact 和 extensible 分别是什么意思?
3. adapter 的 bottleneck 公式是什么?
4. Figure 2 中每个 Transformer layer 为什么有两个 adapters?
5. single adapter 参数量如何从 d 和 m 推出来?
6. near-identity initialization 为什么重要?
7. Table 1 的 80.0 vs 80.4 和 1.3x vs 9x 说明什么?
8. Figure 4 为什么证明只训练 layer norm 不够?
9. Figure 6 的 ablation 说明哪些层更重要?
10. 本地 adapter_original_minimal.py 的输出如何对应论文?

我回答后, 请指出漏洞, 并要求我用 figure/table 或代码函数名对齐。

你也可以让 agent 做反向检查:

请给我一个错误解释, 比如 “adapter 只是训练分类头”,
然后让我指出它哪里错, 并用论文 Figure 2 纠正。

这种练习比让 agent 写摘要更有用。它会把知识压进你的可回忆结构里。

21. 与 Prefix-Tuning 和 LoRA 的关系

和 Prefix-Tuning:

Prefix-Tuning:

- uses trainable key/value-like prefix context
- does not insert bottleneck MLP inside every layer's FFN path

Adapter:

- inserts small bottleneck MLP modules inside the network
- changes hidden activations through residual branch

和 LoRA:

LoRA:

- learns low-rank weight updates
- often merged into base weights for inference

Adapter:

- learns explicit extra modules
- usually stays as separate branch

和 IA3:

IA3:

- learns multiplicative scaling vectors
- fewer parameters

Adapter:

- learns nonlinear bottleneck transformations
- more expressive but heavier

Adapter 的现代意义:

它是 PEFT 的结构化模块路线。后来出现了 Pfeiffer adapters、AdapterFusion、AdapterDrop、Compacter、MAD-X、AdaMix 等变体。

22. 常见误解

误解一: Adapter 只是训练一个分类头。

不对。它在每个 Transformer layer 内部插入 adapter modules, 并训练 layer norm 和 task head。

误解二: Adapter 和 top-layer fine-tuning 一样。

不对。Adapter 在每层提供小的可写分支; top-layer fine-tuning 只修改最后若干层原始参数。

误解三: 参数越少越好。

不对。只训练 layer norm 参数更少, 但 Figure 4 显示性能差。

误解四: Adapter 一开始就大幅改变模型。

不对。论文强调 near-identity initialization。一开始 adapter 应尽量不破坏 pretrained network。

误解五: Adapter 完全没有成本。

不对。它节省存储和训练参数，但推理时要执行额外 adapter modules。

误解六: Adapter 解决 catastrophic forgetting。

更准确地说: 它通过冻结 shared backbone 并保存每任务 adapter，避免旧任务参数被覆盖。但它不是传统 continual learning 中的单模型共享更新方案。

23. 局限性

局限一: 论文主要覆盖分类和抽取式 QA。

它没有像 Prefix-Tuning 那样验证生成任务。

局限二: Adapter 仍增加推理计算。

每层多两个小 MLP，虽然参数少，但 latency 不是零。

局限三: Adapter placement 和规模需要选择。

论文发现若干复杂变体没有明显提升，但这不代表所有模型/任务都一样。

局限四: 每任务仍有 task-specific modules。

任务很多时，虽然远少于 full copies，但 adapter 管理、路由、加载仍是工程问题。

局限五: optional adapters library 环境可能和其他专题冲突。

本仓库当前以 minimal implementations 为主，第三方 adapters 包路径可选。

24. 现代意义

Adapter 论文的长期价值有三点。

第一，它把 PEFT 的工程目标说清楚:

compact
extensible
near full fine-tuning performance

第二，它给了一个非常可复用的设计模式:

frozen backbone
small residual task module
near-identity init

第三，它把评价方式从“最高分”转向“性能-参数量曲线”。

今天你比较 LoRA、Prefix、Adapter、IA3、QLoRA 时，仍然应该问:

- trainable parameters per task 是多少?
- total storage across tasks 是多少?
- 推理额外计算是多少?
- 是否能 batch 多任务?
- 是否能动态加载/卸载任务模块?

- 在小数据和 out-of-domain 下是否稳?

25. 闭卷掌握检查

1. 为什么 full fine-tuning 在 streaming tasks 场景下不 compact?
2. Compact 和 extensible 分别是什么意思?
3. Feature-based transfer、fine-tuning、adapter tuning 三者的区别是什么?
4. Figure 2 中 Houlsby adapter 放在 Transformer 的哪些位置?
5. 单个 adapter 的公式是什么?
6. $2*d*m + d + m$ 是怎么来的?
7. 为什么 Houlsby 每层参数要再乘以 2?
8. Near-identity initialization 为什么重要?
9. 论文除了 adapter 还训练哪些参数?
10. Table 1 的 80.0 vs 80.4 说明什么?
11. Table 1 的 1.3x vs 9x 说明什么?
12. Table 2 为什么能说明它不只是 GLUE trick?
13. Figure 3/4 为什么比单表分数更重要?
14. 为什么只训练 layer norm 不够?
15. SQuAD 实验证明了什么, 没证明什么?
16. Figure 6 中移除单层 adapter 和移除全部 adapter 的差别说明什么?
17. 为什么 lower-layer adapters 影响较小?
18. 本地 adapter_original_minimal.py 中哪个函数对应参数量公式?
19. 本地哪个函数演示 near-identity?
20. Adapter 和 Prefix-Tuning 的机制差别是什么?

26. 最小复述模板

闭卷时可以这样讲:

Houlsby 等人的 Adapter 论文解决的是 BERT 下游任务 full fine-tuning 在多任务场景下参数效率低的问题。如果每个任务都保存一整份 BERT, 任务越多, 总存储越接近 N 份模型。作者提出在 frozen BERT 的 Transformer 层中插入小型 bottleneck adapters。单个 adapter 先把 hidden state 从 d 降到 m , 经过非线性, 再升回 d , 并用 residual 加回输入。Houlsby 结构每个 Transformer layer 放两个 adapters, 一个在 attention 后, 一个在 feed-forward 后。训练时冻结原 BERT, 只训练 adapter、layer norm 和 task head。adapter 初始化要接近 identity, 避免训练开始时破坏 pretrained representation。实验上, Table 1 显示 GLUE 平均分 adapter 80.0, full fine-tuning 80.4, 但总参数约 1.3x 对 9x。Table 2 在额外 17 个分类任务上得到类似结论。Figure 3/4 显示在参数-性能曲线上 adapter 优于只 fine-tune top layers 或只训练 layer norm。Figure 5 说明 SQuAD 也接近 full fine-tuning。Figure 6 说明单层 adapter 影响小、整体影响大, 高层更重要, 初始化过大会伤害训练。这篇的现代意义是: frozen backbone 加小型任务模块, 成为后续 Adapter、Prefix、LoRA、IA3 等 PEFT 方法的共同出发点之一。

如果你能跑通 `adapter_original_minimal.py`, 并解释 25360、608640、near-identity output 和 9-task storage 对比, 这篇就基本进脑子了。